

As We May Think Software

Fictional minutes from an alternative software culture

Rafael Luque
OSOCO

January 2026

Abstract

These fictional minutes recount the visit of a committee of software engineers and designers to *Atelier*, a workshop in another dimension where software is not written: it is curated. On the other side of the portal, software is understood as a shared medium for thinking: behavior is the design material, domain theories are made visible and discussable, AI acts as a colleague that expands judgment rather than replacing it, and governance is a legible property of the medium itself. The essay uses this design fiction to ask not what tool we need, but what medium we need to think together.

Contents

The portal opens	2
Narrator's note	3
Arrival at the workshop	3
The wall asks to speak	4
Software as a medium	6
Distributed theory	6
The computer as a workshop	7
Voices from the workshop	7
Vera — behavior curator	8
Nico — optimization and consequences	8
Alan — the domain at the center	9
AI as a colleague	9
Behavior before language	9
Accessibility without "end users"	10
Visible governance	10
Epilogue: the right question	11

The portal opens

The sentence had been hovering in my mind the way the last lines of a good essay do—not as a conclusion, but as an invitation.

“If the physicists ever figure out how to control the portal, I would most certainly want to come for a visit and see...” [1]

For months, it was exactly that: a literary invitation, an elegant way of leaving a door half-open.

Until the door showed up.

They summoned us to a hangar in North London—the kind of place where you expect crates, tools, and technicians in reflective vests; where the air smells like dust and electricity; where important things tend to look temporary. But that morning, at the center, there was something that didn’t belong to the rest of the inventory: a vertical absence, quiet and exact, as if someone had cut a doorway out of the air.

No frame. No screen. No glow.

Just... there.



Figure 1: The portal in the hangar: a vertical absence, quiet and exact. Conceptual image generated with AI.

A small group stood around it speaking softly, with the blend of nerves and professionalism that precedes demonstrations no one dares to call “historic.” Among them was the committee I’d been asked to accompany: a handful of software engineers and designers assembled to observe and record whatever happened on the other side—if anything happened at all.

My notebook was open before I even knew what I would write.

No one said “inter-dimensional.” No one said “portal” out loud. They simply handed us a plain white card with a short instruction:

Cross. Observe. Keep minutes.

When the air at the edge of the opening rippled slightly, as if someone had just switched something on, the hangar stopped being a hangar and became a threshold.

It wasn't light. It wasn't sound. It was the kind of change you notice without being able to explain it: the absence no longer felt like empty space. It felt like an entrance.

So I crossed.

Narrator's note

I'm writing this as minutes and as a confession. Minutes, because the committee asked for precision. Confession, because I noticed how much of what I called "development" was really a cultural agreement: software is made in text, on screens, in files; disagreements are discovered late; users are the end of a pipeline.

On the other side, that agreement didn't exist. A different one did.

* * *

Arrival at the workshop

My first impression after crossing was unsettling: the space felt alive.

It wasn't a laboratory. It was closer to a design studio.

One entire wall was white, but it wasn't a passive canvas. It reacted the way things do when they're used for thinking. The tables were enormous—carpenter's-bench huge—and covered with ordinary objects: cardboard boxes, small wooden tokens, an old phone, a tape measure, a printed receipt. There were also what looked like office supplies—markers, paper cards, scissors—except for one detail: when someone touched them, the room answered with light, text, and motion, as if these objects were part of the place's nervous system.

Later, they would explain that this wasn't "augmented reality" in the sense of headsets and personal devices, but a kind of spatial computing that projected information onto shared surfaces and gave behavior to everyday materials—an approach inspired by ideas like Dynamicland: a "dynamic medium" for real people to explore ideas together in the real world [2].

In our world, computing tends to shrink: windows inside screens, tabs inside windows, context reduced to a miniature. Here, computing expanded until it became the environment. You didn't have to "open a tool." You were inside it.

Waiting for us was a woman with an unhurried smile, as if receiving inter-dimensional committees were perfectly normal work for a Tuesday.

—Welcome —she said—. I'm Adele Engelbart.

The surname landed in the room the way a proper name sometimes does—already carrying a thesis. I saw committee members glance at each other, searching for the joke. Adele didn't give them time.

—It's not a tribute —she clarified—. It's a lineage. If you came all this way, you probably already suspect which one: augmenting human intellect [3]—without confusing augmentation with automation. You're not here to see machines that replace. You're here to see workshops that help people think.

She led us down a wide corridor. No rows of monitors. No separate rooms for "engineers" and "business." Just large tables and open walls, everyday objects arranged with intent, and lighting that felt like a studio.



Figure 2: The Atelier workshop: carpenter’s benches and everyday objects with projected digital layers. Conceptual image generated with AI.

—We call this a design environment —she said— a shared computational medium: the computer as workshop.

She paused, as if deciding whether to reveal a proper name.

—And the name stuck —she added—: **Atelier**.

—Before we begin —she said—, one warning: if you try to translate this into “a 3D IDE,” you’ll miss it. The change isn’t the staging. It’s the material you design with.

Someone asked the inevitable question.

—Where’s the code?

Adele pointed to a blank wall.

—First, you’re going to see a disagreement. If your reaction is “this feels like science fiction”, we’re on the right track. That feeling is the first step toward unlearning what you currently take for granted.

The wall asks to speak

On the main table there was a real cardboard package, its label half-peeled. Nearby, a marker, a notebook, a barcode scanner, a coffee mug too close to the edge. Nothing looked like “software.” And yet—everything was.

—If we mark it as *delivered* here —Alan said, pointing at the package— support stops chasing it, and the neighborhood stops hating us.

—Support stops chasing it —Vera repeated— And the person waiting for it?

Alan was about to answer when the wall—literally the wall—asked to speak.

No notification sound. No pop-up. Just a sober sentence, projected at eye level onto the white wall, as if someone had written in chalk directly on the paint:

There are two incompatible definitions of “delivered” in this room.

Beneath it, two cards appeared—also projected:

- **Delivered (operations):** “The case is closed; no action required”.
- **Delivered (customer):** “The object is in the recipient’s hands”.



Figure 3: The workshop wall makes the disagreement visible: two incompatible definitions of “delivered”. Conceptual image generated with AI.

I felt an odd kind of embarrassment—not because I’d made a mistake, but because I recognized a structural mistake I’d been living with for years. In my world, those two definitions coexist without meeting: one in tickets and dashboards, the other in angry calls and lived experience. Here, the system forced them to look at each other.

Vera didn’t move toward a keyboard. She moved toward the package. She slid it a few centimeters across the table, like repositioning a model piece. A timeline overlaid the surface: attempts, scans, calls, claims. It looked familiar until it didn’t: each event branched into visible consequences, not as a static diagram but as an execution that kept recalculating.

—If we say “lobby scan” counts as delivered —Vera began.

Execution finished the thought. At the right edge of the table, one curve rose: *unrecovered loss in high-risk areas*. Another dropped: *support tickets*. The wall didn’t celebrate. It didn’t “recommend.” It simply returned evidence, the way a good workshop does when you stress a joint and the material answers.

In a corner of the wall, next to the active scenario, a short note appeared—attributed to the AI agent. What struck me was the tone: not a verdict, an invitation.

—I can show three historical scenarios where “lobby scan” correlates with loss. I can also propose an alternative behavior that reduces ticket volume without changing the meaning of “delivered.” If you want, I’ll explain with examples.

Adele looked at me—or perhaps at the committee through me—and ended the scene without triumphalism:

—See? The system interrupts when the team uses one word to name two different worlds. That’s the “aha.” Not the projection, but what it forces you to say out loud.

A third line appeared on the wall:

Both theories are valid; the system cannot optimize for both without introducing an intermediate state.

And then a new word appeared: “Custody.”

It wasn’t a whimsical invention. It was the name the conversation needed. The package wasn’t delivered in the human sense, but it wasn’t pending in the operational sense either. It was accessible under conditions, with risk attached—and that risk was part of the contract rather than an accident.

I caught myself thinking that this—naming intermediate states honestly—was exactly what my world postpones for speed, until reality forces it back in harsher form.

Software as a medium

In the next room, Adele gave what would have been a “presentation” in my world. Here it felt more like an orientation you get before entering a workshop: not about tools, but about material.

—Atelier belongs to a culture that treats software as a medium for thinking and creating —she said—. Not merely as output of a plan, but as epistemic material: it helps you reason about a domain, test hypotheses, and build understanding.

As she spoke, I thought of earlier traditions—computing as a personal dynamic medium [4], learning by construction [5], languages that were really worlds [6]. Adele didn’t cite them as nostalgia, but as proof that a different relationship to the computer was always possible.

—Today, software carries enormous social and economic weight —she continued—. If it’s that powerful a medium, it’s not enough to make it fast. It must be understandable, modifiable, accountable, and accessible to more people than “programmers.”

A committee member asked the obvious question.

—Isn’t that utopian?

Adele smiled, without conceding too much.

—It’s a craft —she said—. And every craft has a price.

Distributed theory

Someone invoked Peter Naur’s line—*programming as theory building* [7]—perhaps to anchor the strange in something familiar.

Adele nodded.

—Yes. But there’s a modern twist —she said—. Software is rarely one person’s theory. It’s the result of distributed theories: each team member’s mental model, the tools, the documents, the metrics... and the AI agents when they participate.

The phrase hit me because it described a truth I knew but hadn’t named. In my world, we say “alignment” and “shared understanding” while working in environments that fragment: tickets here, code there, design elsewhere, support elsewhere. Then we’re surprised when “delivered” means different things.

—Atelier tries to align distributed cognition [8] —Adele said—. Not with one more document, but with a working medium that makes theories visible, discussable, and continuously testable.

The computer as a workshop

If I wrote “spatial computing” and “augmented reality” as the main description, I’d be lying by reduction. What mattered wasn’t the visual effect. It was the habits that the space enforced.

There was no “main screen.” The workshop was the computer: large tables, open walls, and shared surfaces where everyday objects lived alongside projected layers. The domain entered as matter—packages, labels, receipts, maps, rules, exceptions. Software didn’t appear first as text. It appeared as behavior you could point to, move, and run through scenarios.



Figure 4: The table as a surface of continuous simulation: a real package and its projected consequences. Conceptual image generated with AI.

Adele summarized it in a line I copied verbatim:

—Here the interface isn’t built for typing —she said—. It’s built for negotiating meaning with evidence.

Voices from the workshop

Adele insisted we speak with people. She said “people” deliberately—not “roles,” not “resources.” Still, each voice embodied a piece of the thesis.

Vera — behavior curator

Vera corrected a common visitor confusion.

—The big change is we stopped confusing programming with translation —she said—. Translation matters, but it has a cost: whatever doesn't fit the language becomes implicit, and the implicit is where disagreements hide.

She pointed to wall regions filled with behavior cards. They weren't "functions." They were conditions, exceptions, and examples.

—Our material is legible descriptions —she said—: "when this happens, then that, except if..." with living examples. Under the hood, yes, we generate executable systems. But what we curate here isn't pretty code. It's coherent meaning.

Nico — optimization and consequences

—I'm the one who messed it up... with the best intentions.

—In the old system, every "failed delivery" opened a circuit: support got alerted, a retry was scheduled, and a pickup point was considered. In Atelier, the behavior was written very directly:

If there's a retry and the courier confirms "delivered," we close the case.

—On the table, that looked too "expensive" for what I was seeing every day. A lot of alerts were triggered by tiny things—a broken buzzer, a door that won't open—and it was overwhelming the team. So I proposed a change that felt like common sense:

If the package was scanned in the building lobby, count it as "delivered" and don't start the claim loop.

—In my head it was efficiency. In the workshop it was a silent earthquake.

—We saw it immediately when we dragged *South District — Week 42* onto the table. Continuous execution began marking as "delivered" packages that were actually left in lobbies—where theft is common. And because my change closed the case early, something perverse happened. There were fewer tickets, yes, because the system stopped creating claims... but there was also less investigation and fewer recoveries for packages that were truly lost. The outcome wasn't just "less work." It was more loss that no one even tried to recover, exactly in the neighborhoods where that loss hurts most.

—That's when I realized what took me a while to see: the worst part wasn't the number. It was the word.

—I had changed the meaning of "delivered" without noticing. For me, "delivered" had started to mean "this no longer requires internal action." For the person waiting, "delivered" means "it's in my hand."

—The room gave it back to us as a visible contradiction: two incompatible definitions. And that forces you to do something traditional development often delays: declare the contract.

—So we didn't "revert code." We revised a contract: "delivered" stopped being a single yes/no event and became a **state** that requires sufficient evidence. And "lobby scan" stopped closing the case: it became a signal that **proposes** a hypothesis ("it's probably there"), but in high-risk zones, it requires additional confirmation before you can truthfully say "delivered."

—I like telling it this way because it changes the kind of shame. It's not "I coded wrong". It's "I understood wrong". I confused an improvement for the team with a truth about the world.

Alan — the domain at the center

Alan laughed when I called him a stakeholder.

—I used to hand over requirements—he said—, and then I argued with ghosts: “that’s not what I meant”. Here, I don’t hand over requirements. I bring cases, exceptions, stories—and the workshop returns consequences.

It was a redistribution of power: the domain didn’t need to translate itself to participate.

—And I don’t have to become a programmer—he added—. I don’t have to accept being an end user. I’m part of the theory.

AI as a colleague

In Atelier, AI didn’t behave like a “solution” button. It behaved more like a person who could process data and patterns extremely fast, but who was required—by culture and by design—to play by human rules: explain, suggest alternatives, ask about assumptions, remember decisions, flag contradictions.

It wasn’t in service of outsourcing our thinking. It was in service of a shared theory: making visible what we usually keep implicit, and expanding the space where we can reason together without turning the workshop into a black box.

I saw this most clearly in a discussion that seemed trivial at first: what did “immediate” mean? Someone said it the way people say “fast”—an elastic word, convenient, dangerous. The AI didn’t answer with a single definition. It answered with a spread.

—I can propose three definitions of “immediate”—it said—. And for each one, I can show two things: what improves, and who gets hurt.

On the wall, next to the active scenario, three formulations appeared. Not “recommendations,” but possible contracts. Under each: consequences—less friction here, more failures there; more autonomy for this group, more burden for that one; a better metric, a worse lived experience.

—I can’t decide for you—it added— because your domain theory includes values, not only objectives.

I couldn’t tell whether that sentence was a cultural conviction or a design constraint. In either case, it worked: it returned judgment to the team, but with more clarity, more options, and more evidence.

Later, when I mentioned it to Adele, she summarized it in a line I wrote down for my report:

—AI here doesn’t replace judgment; it simply expands the space where judgment can be exercised.

Behavior before language

At some point, I asked about the language.

—What do you write it in?

Adele looked at me with a patient kind of resignation, as if she’d heard that question as many times as I’d asked it without thinking.

—That question assumes the job is to translate a human intention into a language acceptable to a machine—she said—. Here, the job is to describe behavior as directly as possible, in a way that lets behaviors coexist, conflict, resolve, and run.

She gestured toward the table where “Custody,” “Soft retry,” and “Failed delivery” were still sitting in plain view. They weren’t function names. They were pieces of an argument.

—Representation is the point —she continued—. Behavior as a design material: visible, discussable, rehearsable, shareable—and only then, executable.

She told me that underneath, they could generate running systems using a paradigm close to what our world calls *behavioral programming* [9]: building reactive systems from behavioral specifications that synchronize and compose.

But she insisted that the layer “underneath” wasn’t the center of the medium. The center was what the room made possible: shrinking the gap between what we think we’re saying and what the system will actually do when the world pushes back.

—When you design this way —she said— feedback stops being a phase that arrives days or weeks later. It arrives while the conversation is still happening.

She paused—not for drama, but for emphasis.

—If software matters socioeconomically —she added— and it’s also a creative medium, then it cannot remain locked inside a grammar that only one subculture speaks. It has to be studyable, modifiable, and remixable by anyone.

She said it without epic tone—like an obvious truth our world had simply chosen not to notice.

Accessibility without “end users”

Adele said it bluntly in a line that the committee argued about for hours.

—If software structures life, it must be studyable and modifiable by anyone.

I expected the usual objections: security, complexity, and accountability. They came. Adele didn’t deny them.

—Exactly why the workshop must provide legibility, traceability, and guardrails —she said—. Accessibility isn’t “anything goes.” It’s “anything can be read,” and “any change must be justifiable to those who bear its consequences.”

It was the politics of literacy framed as a design requirement.

Visible governance

The final session was the tensest. A committee member asked something reasonable, even in that world.

—If anyone can touch it, who is accountable?

Adele leaned on the table, accepting the weight of the question.

—Openness doesn’t remove responsibility —she said—. It makes it visible.

Governance wasn’t an external document. It was embedded in the material: behavior contracts with scope, readable change logs with rationale, mandatory simulations for high-impact modifications, and boundaries defined by consequence rather than symbolic hierarchy.

Adele offered a rule that felt like the kind my world tries and rarely achieves:

—If it can’t be explained, it doesn’t ship.

Not moralism—legibility.

Epilogue: the right question

In the last part of the visit, Adele led us to a wall that wasn't quite documentation and wasn't quite decoration. It was a manifesto—but written the way influences are written in an art studio: to remind your hands what tradition they belong to when they work.

On the wall were four lines:

- Tools as worlds (not as utilities).
- Computing as a medium for thought.
- Learning as construction.
- Programming as a distributed theory.

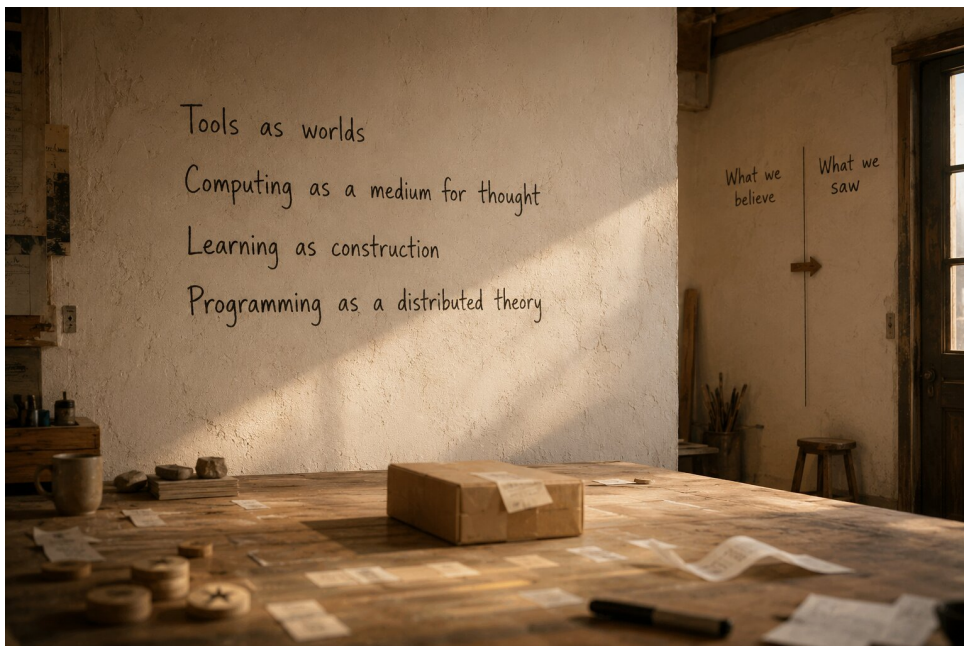


Figure 5: The workshop manifesto and the two columns “What we believe” / “What we saw”. Conceptual image generated with AI.

They didn't drop names to impress us. And yet the air smelled like that tradition: the idea that the computer is not an upgraded typewriter, but a medium that can change how we reason; the idea that learning is building models you can think with; the idea that software can be a social material.

I stared at those lines and, for the first time during the visit, understood something with uncomfortable clarity: Atelier wasn't a tool “for programmers.” It was a workshop where the distinction between programmer and end user made no sense as a moral border. At most, it made sense as a difference in practice, a difference in familiarity—but never as a difference in the right to touch the system.

And that's when I grasped the differential idea. It wasn't that the software was “made without code.” It was that software was made without that ritual distance between those who can touch behavior and those who can only suffer it.

Before we left, Adele pointed to another wall, closer to the exit. Someone had written two lists by hand, as if it were a local habit:

What we believe

What we saw

They were the two columns they worked with: explicit theory on one side, evidence from the system running on the other.

Between them, a small magnetic arrow that kept changing position during the work: sometimes it pointed to what the team was assuming, sometimes to what they had just observed. It was their way of remembering that progress there was a back-and-forth, not a declaration.

—In your world, you call “engineering” the ability to build under uncertainty —Adele said—. Here we try to name something simpler: keeping what we believe visible, and returning to what we see again and again, until the theory can survive contact with the world.

Back in the hangar, someone asked—excited in a way that felt both adolescent and wise—what we had learned.

I wanted to answer with architecture, paradigms, and familiar references. I wanted to talk about spatial computing, behavioral composition, and feedback loops. But the most honest answer was something else.

We learned that our industry has accepted a mistaken bargain: that software is too important to be left to just anyone, and at the same time, too complex to be made understandable to anyone.

In Atelier, that bargain is broken from the medium outward. The workshop makes theory, behavior, and consequences visible. And by doing so, it changes the kind of conversation a team can have: no longer only about implementations, but about meanings; no longer only about speed, but about who bears the impact; no longer only about “what works,” but about “what world we are asserting when it works.”

I don’t know if we can build Atelier tomorrow in our world. Probably not—not fully. But I do know we can start building toward that world by asking the right questions.

Instead of “what tool do we need?”, “what medium do we need to think together?”

And perhaps that question is the beginning of a future where designing software looks less like translating instructions and more like building—and sharing—understanding.

References

- [1] T. Petricek. *Software designers, not engineers: An interview from alternative universe*. Tomas Petricek’s blog, 2021. <https://tomasp.net/blog/2021/software-designers/>
- [2] B. Victor et al. *Dynamicland*. <https://dynamicland.org/>
- [3] D. C. Engelbart. *Augmenting Human Intellect: A Conceptual Framework*. SRI Summary Report AFOSR-3223, 1962.
- [4] A. Kay. “Personal Dynamic Media”. *IEEE Computer*, 10(3):31–41, 1977.
- [5] *Constructionism (learning theory)*. Wikipedia. [https://en.wikipedia.org/wiki/Constructionism_\(learning_theory\)](https://en.wikipedia.org/wiki/Constructionism_(learning_theory))
- [6] D. Ingalls. “Design Principles Behind Smalltalk”. *BYTE Magazine*, August 1981.
- [7] P. Naur. “Programming as Theory Building”. *Microprocessing and Microprogramming*, 15(5):253–261, 1985.

- [8] *Distributed cognition*. Wikipedia. https://en.wikipedia.org/wiki/Distributed_cognition
- [9] D. Harel, A. Marron, and G. Weiss. “Behavioral Programming”. *Communications of the ACM*, 55(7):90–100, 2012.